

Тема 9. Конфигурация проекта и логика событий

process.env, dotenv, .env, EventEmitter и событийная архитектура Node.js

Цель занятия

- Научиться профессионально настраивать Node.js-проект и понимать событийную модель, на которой построены многие API Node.js.
- Понять, зачем выносить настройки из исходного кода.
- Научиться работать с `process.env` и файлом `.env`.
- Подключить `dotenv` в проекте с ES Modules.
- Настроить `package.json` для современного Node.js-проекта.
- Разобрать `EventEmitter` из встроенного модуля `node:events`.
- Научиться создавать собственные события через `on()`, `once()` и `emit()`.
- Реализовать практический пример мониторинга на событиях.

Учебные вопросы

- 1. Безопасность и гибкость: `.env`, `dotenv` и `process.env`.
- 2. Конфигурация `package.json` для ES Modules.
- 3. Логика событий: `EventEmitter` и модуль `node:events`.
- 4. Практика: сервис мониторинга на событиях.
- 5. Запуск проекта через `npm scripts`.

1. Безопасность и гибкость

.env, dotenv и process.env

Главная идея конфигурации

- Исходный код должен быть универсальным, а настройки должны задаваться отдельно от кода.
- Код можно хранить в Git и передавать другим разработчикам.
- Настройки зависят от компьютера, сервера или окружения запуска.
- Пароли, токены, ключи API и строки подключения нельзя записывать прямо в код.
- Если секрет попадёт в публичный репозиторий, его нужно считать скомпрометированным.

Плохо и правильно

Плохо

- Порт записан прямо в коде.
- API-ключ находится в `index.js`.
- Пароль базы данных попадает в Git.
- Для другого сервера нужно менять код.

Правильно

- Код читает настройки из `process.env`.
- Локальные значения лежат в `.env`.
- `.env` добавлен в `.gitignore`.
- Для другого сервера меняется конфигурация, а не код.

Что такое `process.env`

- `process.env` — встроенный объект Node.js, через который приложение получает переменные окружения.
- Переменные окружения задаются операционной системой, терминалом, хостингом или файлом `.env`.
- В коде они доступны как свойства объекта `process.env`.
- Значения из `process.env` приходят как строки.
- Если переменная не задана, результат будет `undefined`.
- Для чисел и булевых значений нужно делать явное преобразование.

Пример чтения process.env

JAVASCRIPT

```
const port = Number(process.env.PORT ?? 3000);  
const nodeEnv = process.env.NODE_ENV ?? 'development';  
  
console.log(`PORT: ${port}`);  
console.log(`NODE_ENV: ${nodeEnv}`);
```

Что важно

- process.env.PORT берётся из окружения.
- ?? 3000 задаёт значение по умолчанию.
- Number(...) преобразует строку в число.
- NODE_ENV часто используют для различения development и production.

Почему нельзя полагаться только на переменные ОС

- На учебном компьютере неудобно вручную задавать переменные перед каждым запуском.
- На Windows, Linux и macOS команды задания переменных отличаются.
- В проекте обычно нужно несколько параметров: порт, ключи, URL базы данных.
- Файл `.env` удобен для локальной разработки.
- На production-сервере переменные обычно задаются средствами хостинга или ОС.

Что такое dotenv

- dotenv — пакет, который читает файл `.env` и добавляет значения из него в `process.env`.
- Файл `.env` хранится в корне проекта.
- Внутри находятся пары ключ=значение.
- dotenv загружает эти пары при старте программы.
- После загрузки код обращается к ним через `process.env`.
- dotenv нужен приложению во время запуска, поэтому устанавливается как обычная dependency.

Установка dotenv

BASH

```
npm install dotenv
```

Что важно

- Пакет добавится в dependencies.
- После установки его можно подключить в index.js.
- В проекте с ES Modules используем import-синтаксис.
- Для Node.js 22 также существует встроенный вариант `node --env-file=.env`, но в этой лекции изучается dotenv как распространённый npm-пакет.

Файл .env

ENV

```
PORT=3000  
SAFE_TEMP=70  
API_KEY=my_secret_key_123  
DB_URL=mongodb://localhost:27017/users_db
```

Что важно

- Каждая строка содержит пару ключ=значение.
- Имена переменных обычно пишут в верхнем регистре.
- Пробелы вокруг знака = лучше не использовать в учебных примерах.
- Реальные секреты нельзя публиковать в Git.

Подключение dotenv в ES Modules

JAVASCRIPT

```
import 'dotenv/config';

const port = Number(process.env.PORT ?? 3000);
const safeTemp = Number(process.env.SAFE_TEMP ?? 70);

console.log(`Сервер будет запущен на порту ${port}`);
console.log(`Безопасная температура: ${safeTemp}°C`);
```

Что важно

- `import 'dotenv/config'` загружает `.env` при старте файла.
- Это ESM-вариант вместо `require('dotenv').config()`.
- `process.env.PORT` и `process.env.SAFE_TEMP` приходят как строки.
- `Number(...)` преобразует значения в числа.

CommonJS и ES Modules для dotenv

CommonJS

- `require('dotenv').config();`
- Используется в старых проектах.
- Работает без `"type": "module"`.
- В этом курсе не используем как основной стиль.

ES Modules

- `import 'dotenv/config';`
- Соплассуется с современным Node.js-кодом.
- Требуёт `"type": "module"` в `package.json`.
- Используется в дальнейших проектах курса.

Файл .gitignore

- .gitignore указывает Git, какие файлы и папки не нужно добавлять в репозиторий.
- .env никогда не должен попадать в публичный Git-репозиторий.
- В .env могут быть пароли, токены и ключи API.
- node_modules тоже не отправляют в Git.
- Вместо настоящего .env создают безопасный шаблон .env.example.

.gitignore

TEXT

```
.env  
node_modules/
```

Что важно

- .env исключается из Git.
- node_modules восстанавливается через npm install или npm ci.
- Так проект можно безопасно передавать другим разработчикам.
- Настоящие секреты остаются только на машине разработчика или на сервере.

.env.example

ENV

```
PORT=3000  
SAFE_TEMP=70  
API_KEY=your_api_key_here  
DB_URL=your_database_url_here
```

Что важно

- .env.example можно отправлять в Git.
- В нём перечислены нужные ключи.
- Реальных паролей и токенов там быть не должно.
- Другой разработчик копирует .env.example в .env и подставляет свои значения.

Итог по конфигурации

1

`process.env` — встроенный объект Node.js для доступа к переменным окружения.

2

`.env` — локальный файл с настройками проекта.

3

`dotenv` загружает `.env` в `process.env`.

4

В ESM-проекте используем `import 'dotenv/config'`.

5

`.env` должен быть в `.gitignore`.

6

`.env.example` помогает документировать нужные параметры без раскрытия секретов.



2. package.json для ES Modules

type, scripts и режим разработки

Зачем здесь нужен package.json

- package.json описывает проект, зависимости, тип модулей и команды запуска.
- Для ES Modules нужно указать "type": "module".
- dependencies и devDependencies фиксируют нужные пакеты.
- scripts позволяют запускать проект короткими командами.
- package.json делает запуск проекта одинаковым на разных компьютерах.

Создание проекта

BASH

```
mkdir node-practice  
cd node-practice  
npm init -y
```

Что важно

- Создаётся папка проекта.
- `npm init -y` создаёт `package.json` со значениями по умолчанию.
- После этого `package.json` нужно отредактировать.
- Для этой лекции добавляем `type`, `scripts` и зависимости.

Установка зависимостей

BASH

```
npm install dotenv  
npm install -D nodemon
```

Что важно

- dotenv нужен приложению во время запуска.
- nodemon нужен только разработчику во время разработки.
- -D записывает пакет в devDependencies.
- Проект можно запускать через npm scripts.

package.json для лекции

JSON

```
{
  "name": "node-practice",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon --watch index.js --watch .env index.js",
    "dev:watch": "node --watch index.js"
  }
}
```

Что важно

- "type": "module" включает import/export в .js-файлах.
- npm start запускает обычный режим.
- npm run dev запускает nodemon.
- npm run dev:watch показывает встроенный watch mode Node.js без nodemon.

nodemon и node --watch

nodemon

- Внешний npm-пакет.
- Удобен для разработки.
- Можно явно указать наблюдение за .env.
- Устанавливается в devDependencies.

node --watch

- Встроенная возможность Node.js.
- Не требует установки пакета.
- Подходит для простых учебных запусков.
- Запускается как `node --watch index.js`.

Итог по package.json

1

`"type": "module"` нужен для ESM-синтаксиса в `.js`-файлах.

2

`dotenv` ставится как `dependency`.

3

`nodemon` ставится как `devDependency`.

4

`scripts` упрощают запуск проекта.

5

`npm start` и `npm run dev` скрывают длинные команды терминала.



3. Логика событий

EventEmitter и модуль node:events

Событийная модель Node.js

- Событийная модель означает, что объект может генерировать событие, а другой код может подписаться на это событие и выполнить реакцию.
- Многие API Node.js построены вокруг событий.
- Файловые потоки генерируют data, end и error.
- HTTP-сервер реагирует на request.
- События позволяют отделить источник события от реакции на него.
- Это делает код гибким и расширяемым.

Издатель и слушатель

- Издатель сообщает, что событие произошло. Слушатель заранее подписывается и реагирует на событие.
- Издатель не обязан знать, кто именно обрабатывает событие.
- Слушатель не обязан управлять внутренней логикой издателя.
- Один объект может генерировать много разных событий.
- На одно событие можно подписать несколько слушателей.

Подключение EventEmitter в ES Modules

JAVASCRIPT

```
import { EventEmitter } from 'node:events';  
  
const emitter = new EventEmitter();
```

Что важно

- node:events — встроенный модуль Node.js.
- EventEmitter импортируется через named import.
- Это ESM-вариант вместо `const EventEmitter = require('events')`.
- emitter — объект, который может создавать события и принимать подписчиков.

Регистрация слушателя через on()

JAVASCRIPT

```
emitter.on('musicStart', (trackName) => {  
  console.log(`Играет трек: ${trackName}`);  
});
```

Что важно

- on() подписывает функцию на событие.
- musicStart — имя события.
- trackName — данные, переданные вместе с событием.
- Слушатель будет срабатывать каждый раз, когда событие произойдёт.

Генерация события через emit()

JAVASCRIPT

```
emitter.emit('musicStart', 'The Beatles - Yesterday');
```

Что важно

- emit() создаёт событие.
- Первый аргумент — имя события.
- Остальные аргументы передаются слушателям.
- Если слушателей нет, событие просто не будет обработано.

Слушатель `once()`

JAVASCRIPT

```
emitter.once('start', () => {  
  console.log('Это сообщение появится только один раз');  
});  
  
emitter.emit('start');  
emitter.emit('start');
```

Что важно

- `once()` подписывает слушателя только на первое событие.
- После первого срабатывания подписка удаляется.
- Второй `emit('start')` уже не вызовет этот listener.
- `once()` удобно использовать для инициализации или одноразовых действий.

Ключевые методы EventEmitter

Метод	Назначение
<code>on(eventName, listener)</code>	Подписывает слушателя на событие
<code>once(eventName, listener)</code>	Подписывает слушателя только на одно срабатывание
<code>emit(eventName, ...args)</code>	Генерирует событие и передаёт данные слушателям
<code>off(eventName, listener)</code>	Удаляет ранее добавленного слушателя
<code>listenerCount(eventName)</code>	Показывает количество слушателей события

Почему события важны для HTTP

- HTTP-сервер в Node.js тоже работает через события.
- Когда приходит запрос, сервер генерирует событие request.
- Код обработчика реагирует на запрос и формирует ответ.
- Не нужно писать бесконечный цикл while(true).
- Node.js ждёт событий от сети, файловой системы, таймеров и потоков.

Идея HTTP-сервера как EventEmitter

JAVASCRIPT

```
server.on('request', (req, res) => {  
  res.end('Hello');  
});
```

Что важно

- request — событие входящего HTTP-запроса.
- req содержит данные запроса.
- res используется для отправки ответа.
- Подробно HTTP-сервер будет изучаться отдельно.

Итог по событиям

1

EventEmitter позволяет объектам общаться через события.

2

on() подписывает слушателя.

3

once() подписывает слушателя на один раз.

4

emit() генерирует событие.

5

ESM-импорт встроенного модуля:
`import { EventEmitter } from 'node:events'.`

6

События лежат в основе многих API Node.js.



4. Практика

Сервис мониторинга на событиях

Практическая задача

- Создать сервис мониторинга, который генерирует случайную температуру и реагирует на перегрев через событие `overheat`.
- Настройки берутся из `.env`.
- `SAFE_TEMP` задаёт безопасный порог температуры.
- `EventEmitter` отделяет наблюдение от реакции.
- `setInterval` имитирует периодическую проверку состояния.
- При перегреве генерируется событие `overheat`.
- При нормальной температуре генерируется событие `status`.

.env для практики

ENV

```
PORT=3000  
SAFE_TEMP=70
```

Что важно

- PORT нужен как пример настройки приложения.
- SAFE_TEMP задаёт предельную безопасную температуру.
- Значения читаются через `process.env`.
- `.env` не добавляется в Git.

index.js: импорт и настройки

JAVASCRIPT

```
import 'dotenv/config';  
import { EventEmitter } from 'node:events';  
  
const port = Number(process.env.PORT ?? 3000);  
const maxTemp = Number(process.env.SAFE_TEMP ?? 70);  
  
const monitor = new EventEmitter();
```

Что важно

- dotenv загружается через ESM-импорт.
- EventEmitter импортируется из node:events.
- PORT и SAFE_TEMP преобразуются в числа.
- monitor будет генерировать события состояния.

index.js: обработчики событий

JAVASCRIPT

```
monitor.on('overheat', (currentTemp) => {  
  console.log(  
    `[ВНИМАНИЕ]: Перегрев! ${currentTemp}°C выше нормы ${maxTemp}°C`  
  );  
});  
  
monitor.on('status', (message) => {  
  console.log(`[LOG]: ${message}`);  
});
```

Что важно

- `overheat` — событие перегрева.
- `status` — обычное информационное событие.
- Реакция на события отделена от логики генерации температуры.
- Если нужно изменить реакцию, не нужно переписывать генератор данных.

index.js: генерация событий

JAVASCRIPT

```
console.log(`Мониторинг запущен на порту ${port}...`);

setInterval(() => {
  const temp = Math.floor(Math.random() * 50) + 40;

  if (temp > maxTemp) {
    monitor.emit('overheat', temp);
  } else {
    monitor.emit('status', `Температура в норме: ${temp}°C`);
  }
}, 2000);
```

Что важно

- setInterval запускает проверку каждые 2 секунды.
- temp имитирует текущую температуру.
- Если temp больше maxTemp, генерируется overheat.
- Иначе генерируется status.

Полный index.js

JAVASCRIPT

```
import 'dotenv/config';
import { EventEmitter } from 'node:events';

const port = Number(process.env.PORT ?? 3000);
const maxTemp = Number(process.env.SAFE_TEMP ?? 70);
const monitor = new EventEmitter();

monitor.on('overheat', (currentTemp) => {
  console.log(`[ВНИМАНИЕ]: ${currentTemp}°C выше нормы ${maxTemp}°C`);
});

monitor.on('status', (message) => {
  console.log(`[LOG]: ${message}`);
});

console.log(`Мониторинг запущен на порту ${port}...`);

setInterval(() => {
  const temp = Math.floor(Math.random() * 50) + 40;
  monitor.emit(temp > maxTemp ? 'overheat' : 'status',
    temp > maxTemp ? temp : `Температура в норме: ${temp}°C`
  );
}, 2000);
```

Что важно

- Весь код использует ES Modules.
- CommonJS require() отсутствует.
- dotenv загружается через import 'dotenv/config'.
- EventEmitter импортируется из node:events.

Запуск проекта

BASH

```
npm run dev
```

Что важно

- Запускает nodemon-режим из package.json.
- Логи должны появляться в консоли каждые 2 секунды.
- При изменении index.js процесс перезапустится.
- При изменении .env nodemon тоже перезапустит процесс, если настроен --watch .env.

Пример вывода

ТЕХТ

```
Мониторинг запущен на порту 3000...  
[LOG]: Температура в норме: 46°C  
[ВНИМАНИЕ]: 87°C выше нормы 70°C  
[LOG]: Температура в норме: 60°C
```

Что важно

- status показывает нормальное состояние.
- overheat показывает превышение порога.
- SAFE_TEMP можно изменить в .env.
- После изменения .env приложение нужно перезапустить.

Выводы

1

Настройки проекта выносятся из кода в `.env`.

2

`process.env` даёт доступ к переменным окружения.

3

`dotenv` загружает `.env` в `process.env`.

4

В ESM-проекте используем `import 'dotenv/config'`.

5

`EventEmitter` импортируется через `import { EventEmitter } from 'node:events'`.

6

`on()` регистрирует реакцию, `emit()` создаёт событие.

7

События помогают разделить логику наблюдения и логику реакции.

8

`scripts` в `package.json` упрощают запуск проекта.

Контрольные вопросы

- 1 Почему нельзя добавлять `.env` в Git-репозиторий?
- 2 Зачем нужен файл `.env.example`?
- 3 Что делает `import 'dotenv/config'`?
- 4 Почему значения из `process.env` нужно преобразовывать в `Number`?
- 5 Зачем в `package.json` добавляют `"type": "module"`?
- 6 В чём разница между `npm start` и `npm run dev`?

- 7 Зачем нужен флаг `-D` при установке `nodemon`?
- 8 Что делает `monitor.on('overheat', listener)`?
- 9 Что делает `monitor.emit('overheat', temp)`?
- 10 Чем `once()` отличается от `on()`?
- 11 Как заставить `nodemon` перезапускать проект при изменении `.env`?
- 12 Почему `EventEmitter` помогает разделить наблюдение и реакцию?